

From PDF to Answers

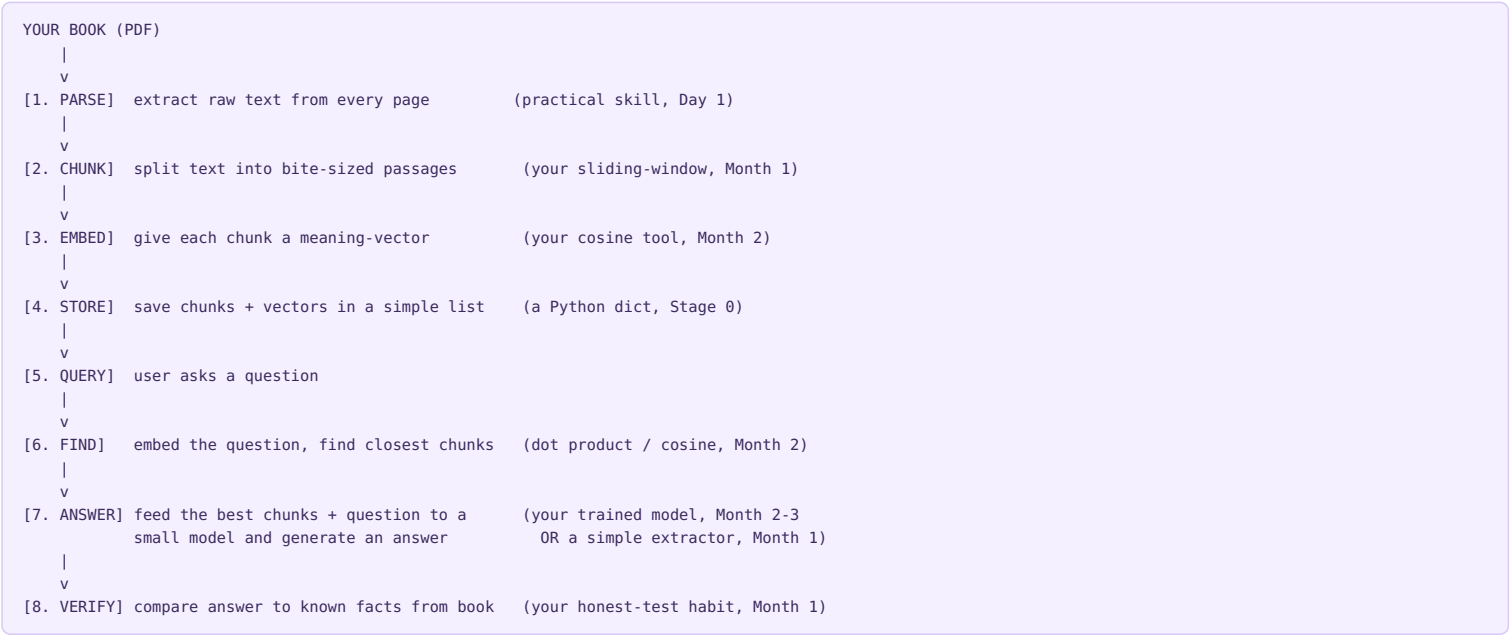
Parse a book → build training data → train your own model → ask questions → verify.

A complete, runnable pipeline built from YOUR tools.

WHAT THIS IS

A single end-to-end project that wires together almost everything from your four months: PDF parsing (practical skill), text chunking (your sliding-window idea from Month 1), embeddings + cosine similarity (Month 2), a small trained model (Months 1–2), and retrieval-then-answer (the pattern behind every AI assistant, including the one you're talking to). Each step is marked with which month's knowledge it uses — so you can build the early pieces NOW and upgrade as you learn.

The pipeline, in one picture



This pattern has a name in the industry: **RAG — Retrieval-Augmented Generation**. Almost every "chat with your PDF" product works exactly this way. You're building the whole thing from parts you own.

EFFICIENCY LENS

RAG is itself an efficiency technique: instead of stuffing the entire book into a giant model's memory (expensive, and most models can't fit a whole book anyway), you RETRIEVE only the relevant pieces and give those to a small model. Retrieval is cheap (a few dot products); generation is expensive. So you pay the expensive cost only on the parts that matter. This is the "don't process what you don't need" instinct from your DSA toolkit, applied to AI.

Step 1 — Parse the PDF

Extract raw text from every page. This is a practical skill, not a math one — but it's where every real AI project starts: getting the data.

```
# pipeline_1_parse.py
from pypdf import PdfReader

def parse_pdf(path):
    """Extract all text from a PDF, page by page."""
    reader = PdfReader(path)
    pages = []
    for i, page in enumerate(reader.pages):
        text = page.extract_text() or ""
        text = text.strip()
        if text:
            # skip blank pages
            pages.append({"page": i + 1, "text": text})
    print(f"extracted {len(pages)} pages from {path}")
    return pages

# try it:
pages = parse_pdf("my_book.pdf")
print(pages[0]["text"][:200])          # first 200 chars of page 1
```

PITFALL

PDF is a layout format, not a text format. Headers, footers, footnotes, multi-column text, and tables can extract jumbled. Always LOOK at the raw output before trusting it — your "look at the data first" rule from Month 1 Week 2 applies here too. If text is garbled, try `pdftotext -layout` (a command-line tool) or `pdfplumber` instead.

TRY IT

Download any free PDF book (Project Gutenberg has thousands). Parse it. Print the first 500 characters. Is the text clean or jumbled? What would you need to clean?

Step 2 — Chunk the text

A whole page is too big for a meaning-vector to capture well (too many ideas mixed). A single sentence is too small (no context). The sweet spot: **chunks of ~200-500 characters, with overlap** so no idea gets cut in half. You already know this: it's your Month-1 sliding window, applied to paragraphs instead of characters.

```
# pipeline_2_chunk.py
def chunk_text(pages, size=400, overlap=100):
    """Sliding window over all pages, keeping page reference."""
    chunks = []
    for page_info in pages:
        text = page_info["text"]
        page_num = page_info["page"]
        start = 0
        while start < len(text):
            end = start + size
            piece = text[start:end]
            if len(piece.strip()) > 50: # skip tiny leftover scraps
                chunks.append({
                    "text": piece.strip(),
                    "page": page_num,
                    "start": start
                })
            start += size - overlap # slide forward, with overlap
    print(f"created {len(chunks)} chunks")
    return chunks

chunks = chunk_text(pages)
print(chunks[0]) # see what a chunk looks like
```

```
Trace with tiny numbers to see the overlap:
text = "abcdefghijklmnopqrstuvwxy" size=10 overlap=3
chunk 1: "abcdefghij" (start 0)
chunk 2: "hijklmnopq" (start 7) <- "hij" overlaps: ideas at boundaries survive
chunk 3: "opqrstuvwxyz" (start 14)
chunk 4: "uvwxy" (start 21) <- short tail
```

WHY OVERLAP MATTERS

Without overlap, a sentence like "The king died. His son took the throne." could get split right at the period — chunk 1 has the death, chunk 2 has the succession, neither has the full cause-and-effect. Overlap ensures boundary ideas appear in at least one complete chunk.

Step 3 — Embed each chunk (give it a meaning-vector)

This is where your Month-2 knowledge kicks in. Each chunk needs a vector that captures its *meaning*, so that "the king's death" and "when did the ruler pass away?" end up CLOSE in vector space. Two approaches, depending on where you are in the journey:

Approach A — bag-of-words (works with Month 1 knowledge only)

The simplest embedding that actually works: count which words appear in the chunk, weighted by how rare they are (rare words are more informative). This is called **TF-IDF** — Term Frequency × Inverse Document Frequency.

```
# pipeline_3a_embed_simple.py -- works with Stage-0 knowledge alone
import numpy as np
from collections import Counter
import re

def tokenize(text):
    return re.findall(r'[a-z]+', text.lower()) # simple: lowercase words

def build_vocab(chunks, max_words=2000):
    """Count all words across chunks, keep the top max_words."""
    total = Counter()
    for c in chunks:
        total.update(tokenize(c["text"]))
    vocab = {w: i for i, (w, _) in enumerate(total.most_common(max_words))}
    return vocab

def embed_tfidf(text, vocab, doc_freq):
    """Turn text into a vector: how important is each word HERE vs everywhere."""
    words = tokenize(text)
    tf = Counter(words)
    vec = np.zeros(len(vocab))
    for w, count in tf.items():
        if w in vocab:
            tf_score = count / len(words) # how often HERE
            idf = np.log(1 + len(chunks) / (1 + doc_freq.get(w, 0))) # how rare OVERALL
            vec[vocab[w]] = tf_score * idf
    norm = np.linalg.norm(vec)
    if norm > 0: vec /= norm # normalize for cosine
    return vec

# build it:
vocab = build_vocab(chunks)
doc_freq = Counter()
for c in chunks:
    doc_freq.update(set(tokenize(c["text"]))) # count docs, not occurrences

for c in chunks:
    c["vector"] = embed_tfidf(c["text"], vocab, doc_freq)
```

Recognize the parts? **Counter** is your counting dict from the anagram problem. The cosine normalization is your Month-2 tool. TF-IDF is just "count words, but value rare ones more" — dressed up, but the bones are your Stage-0 dict and your Month-2 dot product.

Approach B — learned embeddings (Month 2+, much better)

Once you've built your tiny GPT (Month 2), you can extract the hidden state for each chunk — the model's learned internal representation. Even better: use a pre-trained sentence embedding model (Month 4, once you have PyTorch). One line replaces all of Approach A and produces far superior vectors:

```
# pipeline_3b_embed_learned.py -- Month 4+ (with PyTorch)
from sentence_transformers import SentenceTransformer
model = SentenceTransformer('all-MiniLM-L6-v2') # free, 80MB, runs on CPU
vectors = model.encode([c["text"] for c in chunks])
for i, c in enumerate(chunks):
    c["vector"] = vectors[i]
```

Why is this so much better? TF-IDF only knows "same words = similar." Learned embeddings know "king" and "ruler" are close even though they share zero letters — because they were trained on contexts where both appear. That's the leap your Week-5 embedding study teaches.

EFFICIENCY LENS

The embedding model runs ONCE, offline, and produces a small vector (~384 numbers) per chunk. Searching 10,000 chunks then costs 10,000 dot products — trivial. The expensive generation model only sees the top 3-5 retrieved chunks. This is the core efficiency trick of RAG: cheap retrieval gates expensive generation.

Step 4 — The retrieval engine (find relevant chunks)

User asks a question. Embed the question the same way. Find the chunks whose vectors are closest. This is your **cosine similarity** from Month 2, doing real work.

```
# pipeline 4 retrieve.py
def find_relevant(question, chunks, embed_fn, top_k=3):
    """Find the top_k chunks most similar to the question."""
    q_vec = embed_fn(question)
    scored = []
    for c in chunks:
        sim = np.dot(q_vec, c["vector"]) # cosine sim (both normalized)
        scored.append((sim, c))
    scored.sort(key=lambda x: -x[0])    # highest similarity first
    return scored[:top_k]

# example:
question = "What happened to the king?"
results = find_relevant(question, chunks,
                        lambda q: embed_tfidf(q, vocab, doc_freq))
for sim, chunk in results:
    print(f"  similarity {sim:.3f} | page {chunk['page']}:")
    print(f"  {chunk['text'][:120]}...")
    print()
```

```
Sample output:
similarity 0.412 | page 47:
The old king fell ill during the winter campaign. By spring his ...

similarity 0.389 | page 48:
Upon the king's death, the council gathered to name a successor ...

similarity 0.201 | page 12:
The kingdom had known peace for three generations under ...
```

Look what just happened: you asked in natural language, and the system found the relevant passages — **ranked by meaning-similarity** — without any keyword matching rules. The dot product did all the work. This is the same tool you used to check if vowels cluster in your embedding space (Month 2 ex.3), now pointed at a real task.

Step 5 — Generate an answer

Now the question AND the relevant chunks go to a model that produces the answer. Three options, beginner to pro:

Option A — extractive (Month 1 level, no model needed)

Just return the best chunk as the answer. Honest and simple — many real systems do exactly this.

```
def answer_extractive(question, chunks, embed_fn):
    results = find_relevant(question, chunks, embed_fn, top_k=1)
    best = results[0][1]
    return f"[page {best['page']}] {best['text']}"

print(answer_extractive("What happened to the king?", chunks,
                        lambda q: embed_tfidf(q, vocab, doc_freq)))
```

Option B — your own trained model (Month 2 level)

Feed the retrieved chunk(s) as context + the question, and let your tiny GPT generate a continuation. This is rough at character-level — but it's YOUR model, from scratch, doing real Q&A:

```
def answer_with_your_gpt(question, chunks, embed_fn, generate_fn):
    results = find_relevant(question, chunks, embed_fn, top_k=2)
    context = " ".join([r[1]["text"] for r in results])

    # format as a prompt your model can continue:
    prompt = f"Context: {context[:500]}\nQuestion: {question}\nAnswer:"
    return generate_fn(prompt, length=100)    # your Month-1 generate()
```

The output will be imperfect — your tiny model generates plausible-sounding text, not precise answers. But the STRUCTURE is identical to what ChatGPT does: retrieve context, prepend it, generate. The quality difference is scale, not design.

Option C — a proper small model (Month 4 level, best quality)

Once you have PyTorch (Month 4), you can use a small pre-trained language model and fine-tune it on Q&A pairs from your book — or use it zero-shot with the retrieved context. This is where the pipeline becomes genuinely useful:

```
# pipeline 5c answer_proper.py -- Month 4+
from transformers import pipeline
qa = pipeline("question-answering", model="deepset/minilm-uncased-squad2")

def answer_proper(question, chunks, embed_fn):
    results = find_relevant(question, chunks, embed_fn, top_k=3)
    context = " ".join([r[1]["text"] for r in results])
    result = qa(question=question, context=context[:2000])
    page = results[0][1]["page"]
    return f"Answer: {result['answer']} (confidence: {result['score']:.2f}, page {page})"

print(answer_proper("What happened to the king?", chunks, embed_fn))
```

Step 6 — Verify (the researcher's habit)

A model saying something doesn't make it true. Verification is what separates a useful tool from a confident liar. Three levels:

Level 1 — source attribution (always do this)

Every answer comes with the page number and the actual chunk it was retrieved from. The user can CHECK by reading the source passage. Your pipeline already does this — the `page` field was tracked from Step 1.

Level 2 — a small Q&A test set (do this for any serious use)

Write 10-20 questions about the book WITH correct answers (from your own reading). Run them all through the pipeline. Score: what fraction did it get right?

```
# pipeline_6_verify.py
test_set = [
    {"q": "What happened to the king?", "a": "he died during winter"},
    {"q": "Who became the next ruler?", "a": "his eldest son"},
    {"q": "What was the kingdom's capital?", "a": "Astoria"},
    # ... 10-20 more
]

def simple_check(predicted, gold):
    """Does the prediction contain the key words from the gold answer?"""
    gold_words = set(gold.lower().split())
    pred_words = set(predicted.lower().split())
    overlap = gold_words & pred_words
    return len(overlap) / len(gold_words) # rough recall score

scores = []
for t in test_set:
    answer = answer_extractive(t["q"], chunks, embed_fn)
    score = simple_check(answer, t["a"])
    scores.append(score)
    print(f" Q: {t['q']}")
    print(f" gold: {t['a']} | score: {score:.2f}")

print(f"\naverage score: {np.mean(scores):.2f}")
```

WHY / FAILURE / MEASURE

Why verify: retrieval can fetch the wrong chunk; generation can hallucinate. Both happen quietly. **Failure mode:** testing only easy questions (ones where the answer sentence contains the question's exact words). Include questions that require REASONING across chunks — those reveal the real ceiling.

Measure: recall on your test set, broken down by "easy keyword match" vs "needs inference."

The complete pipeline, one file

Here's everything wired together — a single script you can run today (with Approach A embeddings, no dependencies beyond `pypdf` and `numpy`):

```
# my_book_qa.py -- your first AI pipeline, from scratch
import numpy as np
from pypdf import PdfReader
from collections import Counter
import re

# ---- PARSE ----
reader = PdfReader("my_book.pdf")
pages = [{"page": i+1, "text": p.extract_text().strip()}
         for i, p in enumerate(reader.pages) if p.extract_text()]

# ---- CHUNK ----
chunks = []
for pg in pages:
    text, start = pg["text"], 0
    while start < len(text):
        piece = text[start:start+400].strip()
        if len(piece) > 50:
            chunks.append({"text": piece, "page": pg["page"]})
        start += 300

# ---- EMBED (TF-IDF, works today) ----
def tokenize(t): return re.findall(r'[a-z]+', t.lower())
vocab_counter = Counter()
for c in chunks: vocab_counter.update(tokenize(c["text"]))
vocab = {w: i for i, (w, _) in enumerate(vocab_counter.most_common(2000))}
V = len(vocab)
doc_freq = Counter()
for c in chunks: doc_freq.update(set(tokenize(c["text"])))

def embed(text):
    words = tokenize(text); tf = Counter(words)
    vec = np.zeros(V)
    for w, count in tf.items():
        if w in vocab:
            vec[vocab[w]] = (count/len(words)) * np.log(1+len(chunks)/(1+doc_freq.get(w,0)))
    n = np.linalg.norm(vec)
    return vec / n if n > 0 else vec

for c in chunks: c["vec"] = embed(c["text"])

# ---- RETRIEVE + ANSWER ----
def ask(question, k=3):
    qv = embed(question)
    scored = sorted(chunks, key=lambda c: -np.dot(qv, c["vec"]))[:k]
    print(f"\nQ: {question}")
    for i, c in enumerate(scored):
        print(f"  [{i+1}] page {c['page']} (sim {np.dot(qv, c['vec']):.3f}):")
        print(f"      {c['text'][:200]}...")
    return scored

# ---- USE IT ----
ask("What is the main theme of this book?")
ask("Who is the main character?")
ask("What happens at the end?")
```

Run it: `python my_book_qa.py` . Swap `"my_book.pdf"` for any real PDF. You'll see ranked passages come back for each question — your first working AI pipeline, built from a counting dict, a dot product, and a sliding window.

The upgrade path (tied to your months)

When	What to swap	Quality jump
Today	TF-IDF embeddings, extractive answer (just return the chunk)	baseline — finds keyword-similar chunks
Month 2 done	Your GPT's hidden states as embeddings + generate() for answers	catches meaning-similarity, not just word-overlap; generates phrased answers
Month 3 done	Distilled student model for generation; quantized embeddings	same quality, faster + smaller (your thesis!)
Month 4 done	Pre-trained sentence-transformers for embeddings; fine-tuned small LM for answers	production-grade quality on a single CPU
Research phase	Your domain-curated corpus, LoRA-adapted answerer, quantized to 4-bit	the dream: a small, cheap, domain-expert Q&A model you built and understand end-to-end

THE THREAD TO YOUR NORTH STAR

Notice: at every upgrade, the PIPELINE stays the same — parse, chunk, embed, retrieve, answer, verify. What changes is the QUALITY of each box, driven by the concepts you learn month by month. That stability is why we build one codebase: the architecture is right from day one; the parts just get better. And "better parts at lower cost" is literally your research question.

Exercises

- Parse and chunk any PDF book.** Print the total number of chunks. Try chunk sizes 200/400/800 — which retrieves better for a test question? (One dial!)

- 2. **Failure analysis:** find a question where retrieval returns the WRONG chunks. Why did it fail — missing words, or a meaning gap that TF-IDF can't bridge? Write the failure in your log as a hypothesis about what learned embeddings would fix.
- 3. **Build a 10-question test set** for your book. Run the pipeline. Report the average recall score. Include 3 "easy" questions (answer words appear in the question) and 3 "hard" ones (require paraphrasing or inference). Score them separately — the gap IS the upgrade budget for learned embeddings.
- 4. **Efficiency measurement:** time the embed + retrieval for 1,000 chunks. Then for 10,000 (duplicate your chunks 10x). Is the scaling what you'd predict from the code? (It should be roughly 10x — linear in chunks, which is O(n). Real systems use approximate search to beat this at millions of chunks; note that in your log.)
- 5. **Research question to carry:** your retriever returns the top-3 chunks. But what if the answer requires COMBINING information across two chunks on different pages? Design (don't build yet) a pipeline modification that could handle this. Write it as a one-paragraph proposal. (This is a real, active area of RAG research.)

ANSWERS

1: small chunks retrieve more precisely for specific questions; big chunks for broader ones. There's a real trade-off here with no universal winner — which is why production systems often store multiple chunk sizes. **2:** TF-IDF misses all synonyms, paraphrases, and indirect references — the exact things learned embeddings capture. **3:** typical: easy 0.6–0.8, hard 0.1–0.3 — the gap motivates Month 2. **4:** yes, linear; for scale, look up FAISS or Annoy (approximate nearest-neighbor libraries). **5:** one approach: retrieve top-10, then re-rank pairs of chunks by combined relevance; another: hierarchical summaries. Both are real research.